



230968336

SAMEER RAJ

WEEK-5

EXP-4

```
In [1]: import math

maze = [
    [0, 0, 0, 0, 1],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

rows = len(maze)
cols = len(maze[0])

start = (0, 0)
goal = (4, 4)

nodes_expanded = 0

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def successors(state):
    moves = []
    for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
        nx, ny = state[0] + dx, state[1] + dy
        if 0 <= nx < rows and 0 <= ny < cols and maze[nx][ny] == 0:
            moves.append((nx, ny))
    return moves

def rbfs(node, path, g, f_limit):
    global nodes_expanded
    nodes_expanded += 1

    if node == goal:
        return path, g

    successors_list = []
    for s in successors(node):
```

```

        if s not in path:
            f = g + 1 + manhattan(s, goal)
            successors_list.append([s, f])

    if not successors_list:
        return None, math.inf

    while True:
        successors_list.sort(key=lambda x: x[1])
        best = successors_list[0]

        if best[1] > f_limit:
            return None, best[1]

        alternative = successors_list[1][1] if len(successors_list) > 1 else math.inf
        result, best[1] = rbfs(best[0], path + [best[0]], g + 1, min(f_limit, alternative))

        if result is not None:
            return result, best[1]

path, cost = rbfs(start, [start], 0, math.inf)

print("Path:", path)
print("Path Cost:", cost)
print("Nodes Expanded:", nodes_expanded)

```

Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
 Path Cost: 8
 Nodes Expanded: 9

EXP-5

In [4]: **import** math

```

maze = [
    [0, 0, 0, 0, 1],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

rows = len(maze)
cols = len(maze[0])

start = (0, 0)
goal = (4, 4)

nodes_expanded = 0

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

```

```

def successors(state):
    moves = []
    for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
        nx, ny = state[0] + dx, state[1] + dy
        if 0 <= nx < rows and 0 <= ny < cols and maze[nx][ny] == 0:
            moves.append((nx, ny))
    return moves

def dfs(path, g, threshold):
    global nodes_expanded
    node = path[-1]
    f = g + manhattan(node, goal)

    if f > threshold:
        return f
    if node == goal:
        return path

    min_cost = math.inf
    nodes_expanded += 1

    for s in successors(node):
        if s not in path:
            result = dfs(path + [s], g + 1, threshold)
            if isinstance(result, list):
                return result
            if result < min_cost:
                min_cost = result

    return min_cost

threshold = manhattan(start, goal)

while True:
    result = dfs([start], 0, threshold)
    if isinstance(result, list):
        print("Path:", result)
        print("Path Cost:", len(result) - 1)
        print("Nodes Expanded:", nodes_expanded)
        break
    if result == math.inf:
        print("No solution found")
        break
    threshold = result

```

Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
 Path Cost: 8
 Nodes Expanded: 8

EXP-6

```
In [3]: import math

maze = [
    [0, 0, 0, 0, 1],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

rows = len(maze)
cols = len(maze[0])

start = (0, 0)
goal = (4, 4)

MEMORY_LIMIT = 20
nodes_expanded = 0

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def successors(state):
    result = []
    for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
        nx, ny = state[0] + dx, state[1] + dy
        if 0 <= nx < rows and 0 <= ny < cols and maze[nx][ny] == 0:
            result.append((nx, ny))
    return result

class Node:
    def __init__(self, state, parent, g):
        self.state = state
        self.parent = parent
        self.g = g
        self.h = manhattan(state, goal)
        self.f = self.g + self.h

def reconstruct(node):
    path = []
    while node:
        path.append(node.state)
        node = node.parent
    return path[::-1]

open_list = []
open_list.append(Node(start, None, 0))

while open_list:
```

```

open_list.sort(key=lambda n: n.f)
current = open_list.pop(0)
nodes_expanded += 1

if current.state == goal:
    path = reconstruct(current)
    print("Path:", path)
    print("Path Cost:", len(path) - 1)
    print("Nodes Expanded:", nodes_expanded)
    break

for s in successors(current.state):
    child = Node(s, current, current.g + 1)
    open_list.append(child)

    if len(open_list) > MEMORY_LIMIT:
        open_list.sort(key=lambda n: n.f)
        removed = open_list.pop()
        if removed.parent:
            removed.parent.f = min(removed.parent.f, removed.f)
else:
    print("No solution found")

```

Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]

Path Cost: 8

Nodes Expanded: 10

In []: